

Hackathon Brief

Hackathon Brief: Keeping a Self-Improving LLM Validator Honest and Cost-Effective

Format: Half-day hackathon, teams of 2-4

1. Background

Most teams that use an LLM to validate or QA data run it the same way every time: same prompt, same data, no memory of what happened before. When the model gets something wrong, a human eventually catches it — but that correction dies with the conversation. The same mistake happens again next time.

One team is fixing this with a feedback loop:

1. LLM validates a record.
2. If it's not confident, the record is routed to a human reviewer.
3. The human reviews it and leaves a comment.
4. The feedback is sent back to an LLM, which synthesizes a **new rule** and adds it to a growing rule bank.
5. Future runs include relevant rules alongside the base prompt.

This works — for a while. But it creates a second problem, which is what this hackathon is about.

2. The Problem You're Solving

A rule bank that only grows, and gets fully replayed into every call, breaks in two ways as it grows:

Quality breaks down: - You can't tell which rules are pulling weight, vs. which are dead weight the base prompt was already handling. - Rules overlap and contradict each other, which increases hallucination risk rather than reducing it. - Stale rules linger long after they stop being relevant.

Cost breaks down: - If every validation call stuffs the *entire* rule bank into context, cost and latency grow as the rule bank grows, indefinitely. - Image inputs (relevant to today's domain) already make individual calls more expensive — so "always run the full model on everything" gets expensive fast at real volume.

Your challenge: build a system that (1) continuously tells you whether each rule is earning its place, and (2) keeps per-call cost roughly flat as the rule bank and data volume both grow. Human review stays in the loop for all rule decisions — your system's job is to surface evidence, not to auto-edit anything.

3. Domain: Shop-to-Shop Match Validation

Task: given a shop from Shopee (name + icon) and a candidate shop from a competitor platform (Lazada or TikTok Shop — also name + icon), decide whether they are **the same seller/brand**, and produce a verdict a human could sanity-check quickly.

Suggested output per pair (define your own, but be explicit and consistent): - A verdict: Match / Likely Match / Likely Not a Match / Not a Match / Uncertain - A short rationale (1–2 sentences) - Which signals it relied on (name similarity, icon similarity, both, neither)

This is a genuine matching problem — the hard cases are things like: - Same brand, different sub-brand naming across platforms ("Nike Official Store" vs "Nike Flagship Store") - Rebrands or seasonal name changes - Coincidental name overlap, unrelated seller ("Sunshine Mart" on Shopee vs. an unrelated "Sunshine Mart" on Lazada) - Same name, visually different icon (compromised, rebranded, or copycat account) - Same or similar icon, different name (multi-storefront sellers)

4. Build Your Own Dataset

You'll build your own labeled dataset — there's no starter dataset provided.

Minimum bar for a usable dataset: - **At least 20 pairs**, each labeled Match or Not a Match by your own team acting as the human reviewer — write down *why* for each. That rationale is your "reviewer comment" for later steps. - Deliberately include at least: - A few genuine matches with different names across platforms (rebrand/sub-brand style) - A few genuine matches with visually different icons - A few confusable non-matches (similar name, unrelated seller) - A few confusable non-matches (similar icon style/colors, different brand) - Icons don't need to be scraped from live platforms — placeholder/mock logos are fine. What matters is the *pattern* (similar-but-different, identical-but-unrelated), not photorealism. - Once built, treat this dataset as fixed. It's your held-out evaluation set for the rest of the day — don't let it grow or drift as you build.

Building a lean, well-chosen dataset matters more than a large one. How you allocate your limited time across this step vs. the rest of the build is part of what's being evaluated.

5. Design Constraints

Work within these — but you're welcome to push back on any of them in your final presentation if you have a better idea and evidence for it.

- **Cost-effective first, expensive second.** Don't send every pair straight to a vision-capable LLM call with the full rule bank attached. Use a cost-effective, fast pre-filter first (e.g., simple name string similarity + a lightweight image comparison), and only escalate genuinely ambiguous pairs to full LLM judgment. This is the single biggest lever on cost at scale.
- **The rule bank should be retrieved, not replayed.** As the rule bank grows, don't inject all rules into every call. Retrieve only the rules relevant to the current pair (e.g., by similarity between the pair's signals and each rule's description) and inject just those. This is what keeps cost roughly flat as the rule bank grows.
- **LLM confidence is not trustworthy by default.** Don't just ask

“how confident are you, 0-1?” and trust the number. Use structured reasoning before the score, or sample the same pair more than once and use agreement as a proxy.

- **Your labeled dataset is your evaluation set — permanently.** Every pair you hand-label is ground truth going forward. Don’t discard it once you start building rules from it.
 - **Rule impact must be measured per-rule, not per-batch.** “Accuracy with all rules on” vs. “all rules off” tells you nothing about one specific rule. Ablate rules individually against your fixed evaluation set.
 - **No rule goes live, gets merged, or gets deprecated without a human decision.** The system recommends; a person approves.
 - **Separate the matcher from the maintenance harness.** The part that compares a shop-name/icon pair is domain-specific. The part that scores rules, retrieves relevant rules, and flags conflicts/staleness should be domain-agnostic — the same harness should be reusable on a completely different matching problem later without a rewrite.
-

6. Suggested Build Tiers

Use these as checkpoints for yourselves, not a mandated sequence. Pick a tier honestly based on how the day is going — a solid, working must-have beats a half-finished stretch goal.

Must-have

- Your labeled dataset (Section 4).
- A working matcher: cost-effective pre-filter (name/icon similarity) → LLM judgment only on ambiguous pairs → verdict + rationale.
- A **rule scorecard**: for each rule in your rule bank, measure accuracy with vs. without that rule on your evaluation set, and flag any rule with near-zero measurable lift.

Should-have

- **Selective rule retrieval**: instead of injecting the whole rule bank into every call, retrieve only the top few relevant rules per pair. Show that this doesn’t meaningfully hurt accuracy vs. full injection, while cutting tokens per call.
- A rough **cost estimate**: tokens (and roughly, cost) per validation call at your current design, and how that projects if volume were 100x and the rule bank were 10x. Order-of-magnitude is fine — precision isn’t the point.

Stretch

- Conflict/overlap detection across rules (cluster similar rules, propose merges for a human to approve).
- A single view showing scorecard + retrieval hit-rate + estimated cost per call together, so a reviewer sees maintenance signal and cost signal side by side.

Out of scope for today: scraping large volumes of real shop listings, fine-tuning, or a fully general system built to handle every possible matching domain. One clean, working pipeline on your own dataset is the goal.

7. Deliverables

Working code is preferred, but not required. A clear, well-reasoned thought process — design decisions, tradeoffs, and how you’d validate them — is an acceptable substitute for a fully running pipeline. What matters most is the quality of your thinking; a working demo is a bonus, not a gate.

Come prepared to show, in whichever form you’ve built it (running code, a worked example by hand, or a clearly laid-out design):

1. Your dataset and the reasoning behind the 2–3 trickiest pairs you included.
2. How the matcher would resolve a few pairs — including at least one that resolves through the cost-effective filter alone, and one that escalates to the LLM.
3. The rule scorecard (or your reasoning for what it would show), with at least one concrete recommendation (e.g., “rule X shows no measurable lift on our evaluation set — consider retiring”).
4. One sentence each on: how your design would change at 100x volume, and what part of it is reusable for a completely different matching problem.

8. Judging Criteria

Criterion	What we’re looking for
Dataset quality	Are the hard/confusable cases actually present, not just easy ones?
Time & scope management	Did the team allocate effort sensibly across dataset, matcher, and maintenance harness, and land on a well-reasoned result?
Cost-consciousness	Is there a real cost-effective-first escalation path, or does every pair hit the expensive path?
Methodological soundness	Is rule impact isolated per-rule against a fixed evaluation set?
Human-in-the-loop preserved	Does the system recommend, or does it quietly auto-edit the rule bank? (Auto-edit counts against you.)
Transferability	Could the maintenance harness be reused if the matcher were swapped for a different domain?
Communication	Can the team name a failure mode in their own design, unprompted?

Note: working code is preferred but not required — a well-argued design and thought process scores on the same criteria as a working build.

9. A Closing Thought

At real volume, “keep adding rules to every prompt” eventually starts to look like what fine-tuning or a proper retrieval index is meant to solve more cleanly. What you’re building today is meant to keep a growing rule bank *honest and cost-effective* in the meantime — not to

be the permanent architecture forever. Worth keeping in mind so the design doesn't quietly drift toward "bigger rule bank, stuffed into every call" as the assumed end state.